

Docu-Intel

Arquitectura ideal para OCR documental, RAG seguro y consulta por presupuesto

Fecha: 15/05/2026

Entorno objetivo: servidor remoto por VPN + servidor app en Coolify + FastAPI + PostgreSQL + Qdrant + workers OCR/IA

Decision principal

La otra app nunca accede a archivos, PostgreSQL, Qdrant ni al servidor de embeddings. Solo puede consultar FastAPI. Cada carpeta de presupuesto se representa como un `budget_scope_id` y todas las consultas se fuerzan a ese scope.

Resumen de la solucion

- Servidor remoto: fuente original intacta; solo lectura; nunca borrar, mover ni modificar.
- Servidor app: copia local, OCR, extraccion, embeddings, Qdrant, PostgreSQL y auditoria.
- Aislamiento: API key + presupuesto permitido + filtro obligatorio en Qdrant por `budget_scope_id`.
- Planos gigantes: preview + tiles con solape + OCR por zona + VLM solo en tiles relevantes.
- Qdrant self-hosted: gratuito bajo licencia Apache 2.0; Qdrant Cloud tiene free tier limitado.

1. Objetivo

Construir un sistema que permita consultar con IA externa la informacion de un presupuesto concreto sin contaminarse con datos de otros presupuestos. El sistema debe procesar carpetas no uniformes con PDFs, Excels, Word, imagenes, emails y planos, generar OCR y embeddings, y exponer una API segura para RAG.

Regla base de seguridad

El presupuesto no lo decide la IA. Lo decide el backend despues de validar API key, permiso sobre el presupuesto y filtro obligatorio de busqueda.

2. Arquitectura general

SERVIDOR REMOTO - solo lectura

/ruta/presupuestos/

├─ 245745/

├─ 484857/

└─ ...

VPN + rsync sin --delete

↓

SERVIDOR APP / COOLIFY

/srv/docuintel/

├─ inbox/ copia local desde remoto

├─ processing/ carpetas en proceso OCR

├─ files/ originales definitivos por SHA256

├─ quarantine/ archivos sospechosos o invalidos

├─ failed/ fallos

├─ exports/ posibles exportaciones

└─ logs/ logs locales

↓

FastAPI + PostgreSQL + Qdrant + Redis/Celery + OCR + Embeddings

↑

| API segura

↓

OTRA APP / CHAT IA

- No accede al servidor remoto

- No accede a archivos

- No accede a PostgreSQL

- No accede a Qdrant

- Solo llama a /api/v1/query

3. Principios absolutos

Regla	Aplicacion
Origen intacto	El servidor remoto nunca se borra, mueve ni modifica. Solo se lee mediante rsync.
Sin acceso directo	La otra app no toca archivos, PostgreSQL, Qdrant ni embeddings.
Scope cerrado	Cada carpeta/presupuesto se convierte en un budget_scope_id.
Filtro obligatorio	Toda consulta vectorial incluye budget_scope_id en Qdrant.
Permiso por API key	La API key debe tener permiso explicito sobre ese

Regla	Aplicacion
	presupuesto.
Auditoria	Toda pregunta, cliente API y chunks devueltos se registran.
Fuentes obligatorias	La respuesta siempre devuelve documento, pagina, chunk y score.

4. Stack recomendado

Capa	Tecnologia	Funcion
API	FastAPI	Unica puerta de entrada para consultas y gestion.
Metadata DB	PostgreSQL 16	Presupuestos, documentos, permisos, jobs, errores y auditoria.
Vector DB	Qdrant self-hosted	Vectores, chunks y payload filtrable por budget_scope_id.
Colas	Redis + Celery	Procesamiento asincrono por tipo de carga.
OCR	PaddleOCR	OCR de PDFs escaneados, imagenes y tiles de planos.
Extraccion	pypdf, openpyxl, python-docx, email parser	PDF digital, Excel, Word, EML, TXT.
Embedding texto	BAAI/bge-m3	Embedding principal potente para OCR y documentos.
Embedding ligero	nomic-embed-text-v2-moe	Alternativa mas liviana si el servidor va justo.
VLM	Qwen2.5-VL 7B	Interpretacion de imagenes, diagramas y zonas de planos.
Deploy	Docker + Coolify	Despliegue y redes aisladas.

5. Estructura fisica de carpetas

```

/srv/docuintel/
├─ inbox/
│  └─ 245745/
│  └─ 484857/
│  └─ ...
├─ processing/
│  └─ 245745/
├─ files/
│  └─ sha256/
│     └─ ab/
│        └─ cd/
│           └─ abcd1234....pdf
├─ previews/
│  └─ document_id/page_1_preview.jpg
├─ tiles/
│  └─ document_id/page_1/tile_0001.jpg
├─ quarantine/
├─ failed/
└─ models/

```

```

├─ exports/
├─ logs/

```

- No procesar directamente desde inbox.
- Mover la copia local a processing antes de procesar.
- Guardar cada original valido en files/sha256 por hash.
- Limpiar temporales locales solo despues de indexar y registrar correctamente.
- El remoto siempre queda intacto.

6. Servicios Docker / Coolify

Servicio	Funcion	Red
backend	FastAPI, autenticacion, permisos, query RAG, respuesta con fuentes.	public + private
worker-fast	PDF digital, DOCX, XLSX, TXT, EML.	private
worker-ocr	OCR pesado, imagenes, PDFs escaneados.	private
worker-plan	Planos gigantes, tiles, OCR por zona.	private
worker-embedding	Generar vectores y upsert en Qdrant.	private
sync	rsync desde remoto hacia inbox local.	private
postgres	Metadatos, permisos, auditoria, estados.	private
qdrant	Vector DB y busqueda por payload.	private
redis	Broker Celery.	private
embedding-server	Servidor local de embeddings.	private

Redes Docker

Qdrant, PostgreSQL, Redis y embedding-server deben estar solo en red privada. Solo backend expone API hacia la otra app.

7. PostgreSQL - esquema ideal

PostgreSQL es la fuente de verdad administrativa. Qdrant no sustituye a PostgreSQL: Qdrant solo indexa vectores y payload filtrable.

Tabla	Responsabilidad
budget_scopes	Una fila por presupuesto/carpeta principal.
documents	Archivos originales, hashes, rutas, estados de OCR y embeddings.
document_chunks	Chunks generados y relacion con puntos de Qdrant.
document_entities	Importes, medidas, proveedores, habitaciones, referencias.
plan_pages	Paginas de planos, dimensiones, preview, DPI.
plan_tiles	Tiles de planos gigantes con coordenadas.
plan_ocr_blocks	Bloques OCR con bbox dentro del tile.
api_clients	Clientes autorizados/API keys.
api_client_budget_scopes	Permisos de cada API key por presupuesto.
ingest_jobs	Jobs de ingesta, OCR, embeddings, reprocesado.
document_errors	Errores por archivo/procesador.
query_audit_logs	Auditoria de consultas externas.

7.1 Tablas principales - SQL base

```

CREATE TABLE budget_scopes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  presupuesto_code VARCHAR(80) UNIQUE NOT NULL,
  remote_folder TEXT,
  local_inbox_folder TEXT,
  display_name TEXT,
  status VARCHAR(30) DEFAULT 'pending',
  total_files INTEGER DEFAULT 0,
  processed_files INTEGER DEFAULT 0,
  failed_files INTEGER DEFAULT 0,
  last_sync_at TIMESTAMP,
  last_processed_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE documents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  budget_scope_id UUID NOT NULL REFERENCES budget_scopes(id),
  original_filename TEXT NOT NULL,
  remote_path TEXT,
  local_inbox_path TEXT,
  relative_path TEXT,
  storage_path TEXT,
  extension VARCHAR(20),
  mime_type VARCHAR(100),
  file_type VARCHAR(50),
  sha256 VARCHAR(64) NOT NULL,
  size_bytes BIGINT,
  status VARCHAR(30) DEFAULT 'pending',
  extraction_status VARCHAR(30) DEFAULT 'pending',
  ocr_status VARCHAR(30) DEFAULT 'not_required',
  embedding_status VARCHAR(30) DEFAULT 'pending',
  page_count INTEGER,
  text_chars INTEGER,
  needs_review BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE UNIQUE INDEX idx_documents_budget_sha
ON documents(budget_scope_id, sha256);
CREATE TABLE document_chunks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  budget_scope_id UUID NOT NULL REFERENCES budget_scopes(id),
  document_id UUID NOT NULL REFERENCES documents(id),
  qdrant_point_id UUID,
  chunk_index INTEGER NOT NULL,
  page_number INTEGER,
  text_preview TEXT,
  token_count INTEGER,
  source_type VARCHAR(50),
  confidence NUMERIC(5, 4),
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE api_clients (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  api_key_hash TEXT NOT NULL,

```

```

    active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT NOW()
);

```

```

CREATE TABLE api_client_budget_scopes (
    api_client_id UUID REFERENCES api_clients(id),
    budget_scope_id UUID REFERENCES budget_scopes(id),
    can_query BOOLEAN DEFAULT TRUE,
    can_see_amounts BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT NOW(),
    PRIMARY KEY (api_client_id, budget_scope_id)
);

```

8. Qdrant - estructura vectorial

Usar una sola collection y separar presupuestos mediante payload. No crear una collection por presupuesto.

Collection: docu_intel_chunks
 Vector size con BGE-M3: 1024
 Vector size con Nomic v2 MoE: 768
 Distance: Cosine

Payload obligatorio	Uso
budget_scope_id	Filtro principal obligatorio.
presupuesto_code	Codigo legible, por ejemplo 245745.
document_id	Relacion con PostgreSQL.
chunk_id	Relacion con document_chunks.
file_type	pdf, xlsx, docx, eml, image, plan.
relative_path	Fuente dentro de la carpeta.
page_number	Pagina de origen.
tile_id / bbox	Solo en planos gigantes.
text	Texto del chunk usado para RAG.
contains_amounts	Permite redactar importes si no hay permiso.

Filtro obligatorio de busqueda:

```

{
  "filter": {
    "must": [
      {"key": "budget_scope_id", "match": {"value": "uuid-del-presupuesto"}}
    ]
  },
  "limit": 8
}

```

9. API segura para la otra app

Endpoint	Funcion
GET /api/v1/health	Estado del sistema.
POST /api/v1/query	Consulta RAG limitada a un presupuesto.
POST /api/v1/ingest/budget	Crear/registrar presupuesto.
POST /api/v1/ingest/budget/{code}/scan	Escanear una carpeta ya copiada.
GET /api/v1/budgets/{code}/status	Estado del presupuesto.
GET /api/v1/ingest/jobs/{job_id}	Estado de job.

Endpoint	Funcion
POST /api/v1/admin/reprocess/document/{id}	Reprocesar documento.

Request:

POST /api/v1/query

X-API-Key: xxxxx

X-Presupuesto-Id: 245745

Content-Type: application/json

```
{
  "pregunta": "¿Cuántas habitaciones aparecen para cortinas?",
  "top_k": 8,
  "include_sources": true
}
```

Flujo interno:

1. Verificar X-API-Key.
2. Resolver api_client_id.
3. Leer X-Presupuesto-Id.
4. Resolver presupuesto_code -> budget_scope_id.
5. Comprobar api_client_budget_scopes.
6. Generar embedding de la pregunta.
7. Buscar en Qdrant con filtro obligatorio por budget_scope_id.
8. Redactar importes si can_see_amounts = false.
9. Generar respuesta con fuentes.
10. Registrar query_audit_logs.

10. Sincronizacion desde servidor remoto

Regla definitiva

El script de sync nunca borra ni modifica el origen. No usar --delete, rm -rf ni mv en el servidor remoto.

```
#!/bin/bash
set -euo pipefail

REMOTE_SERVER="usuario@servidor-remoto"
REMOTE_PATH="/ruta/presupuestos/"
LOCAL_PATH="/srv/docuintel/inbox/"

mkdir -p "$LOCAL_PATH"
```

```
rsync -avz \
  --partial \
  --append-verify \
  --human-readable \
  "$REMOTE_SERVER:$REMOTE_PATH" \
  "$LOCAL_PATH"
```

echo "Sync completado. No se ha borrado ni modificado nada en origen."

11. Pipeline de ingesta

1. rsync copia desde remoto a /srv/docuintel/inbox/.
2. El scanner detecta carpeta de presupuesto, por ejemplo 245745.
3. Se crea o actualiza budget_scope.
4. La copia local se mueve a /processing/245745/.
5. Scanner recursivo localiza todos los archivos, aunque las subcarpetas no sean uniformes.
6. Preflight: archivo estable, extension permitida, MIME real, tamaño maximo, SHA256.

7. El original valido se guarda en /files/sha256/ab/cd/hash.ext.
8. Se registra documents en PostgreSQL.
9. Se extrae texto por tipo: PDF digital, OCR, Excel, Word, EML, imagen, plano.
10. Se generan chunks de 400-700 tokens con overlap 80-120 tokens.
11. Se generan embeddings.
12. Se hace upsert en Qdrant con budget_scope_id.
13. Se actualizan estados y auditoria.
14. Se limpia processing local. El servidor remoto queda intacto.

12. Procesadores por tipo documental

Tipo	Procesamiento recomendado
PDF digital	Extraer texto con pypdf/PyMuPDF; OCR solo si falta texto.
PDF escaneado	Render por pagina o por tiles; PaddleOCR.
Excel	openpyxl; preservar hojas, filas, columnas y valores.
Word	python-docx; extraer parrafos y tablas.
EML	Extraer asunto, remitente, fecha, cuerpo y adjuntos.
Imagen	OCR + descripcion opcional con VLM.
Plano gigante	Preview + tiles + OCR por zona + VLM solo en zonas relevantes.
DWG/DXF	Convertir a PDF/SVG/imagen; para DXF intentar extraer entidades tecnicas.

13. Planos gigantes

Los planos A0/A1, TIFF pesados, imagenes de muchos megapixeles o PDFs vectoriales grandes no deben mandarse enteros al VLM. Se procesan por zonas.

- Plano gigante
- > detectar tamaño, paginas, DPI y tipo
 - > crear preview general
 - > dividir en tiles 2048x2048 o 3072x3072
 - > solape 10%-15%
 - > OCR por tile
 - > guardar texto + coordenadas
 - > embeddings por zona
 - > Qdrant con budget_scope_id + tile_id + bbox
 - > VLM solo en tiles relevantes
 - > respuesta con fuente, pagina y coordenadas

Parametro	Recomendacion
Tile size	2048x2048 o 3072x3072 px.
Solape	10%-15% para no cortar textos en bordes.
DPI	200-300 DPI; 400 solo para zonas con texto pequeno.
Concurrencia	worker-plan con concurrency=1.
Medidas	Leer medidas escritas; no calcular geometricamente salvo escala fiable.

Tablas adicionales:

- plan_pages: pagina, ancho, alto, DPI, preview_path.
- plan_tiles: x, y, width, height, overlap_px, tile_image_path.
- plan_ocr_blocks: texto, bbox, confidence, block_type.

block_type sugeridos:
dimension, room_name, reference, material, note, symbol, scale, legend.

14. Modelos recomendados

Funcion	Modelo recomendado	Notas
Embedding principal	BAAI/bge-m3	Potente, multilingue, 1024 dims, documentos largos.
Embedding ligero	nomic-embed-text-v2-moe	Mas liviano, 768 dims, GGUF, bueno para MVP.
VLM principal	Qwen2.5-VL 7B	Bueno para documentos, diagramas, tablas, layouts e imagenes.
OCR	PaddleOCR	OCR potente, multilingue, adecuado para PDFs/imagenes.
Visual embedding avanzado	Jina Embeddings v4	Opcional para retrieval multimodal de documentos visuales.
Reranker opcional	BAAI/bge-reranker-v2-m3	Reordena resultados antes de pasar contexto a la IA.

Recomendacion final de modelos

Produccion equilibrada: BGE-M3 + Qdrant + PaddleOCR + Qwen2.5-VL 7B. Si el servidor va justo: Nomic v2 MoE + Qwen2.5-VL 3B.

15. Tests obligatorios de seguridad

- test_sync_never_deletes_remote
- test_query_without_api_key_rejected
- test_query_without_presupuesto_id_rejected
- test_api_key_cannot_access_unassigned_budget
- test_question_mentions_other_budget_but_scope_stays_current
- test_qdrant_filter_always_contains_budget_scope_id
- test_other_app_cannot_reach_qdrant
- test_other_app_cannot_reach_postgres
- test_response_always_contains_sources
- test_amounts_redacted_when_not_allowed

Caso critico:

Header: X-Presupuesto-Id: 484857

Pregunta: "Dame informacion del presupuesto 245745"

Resultado correcto:

- Se consulta solo budget_scope_id de 484857.
- Se registra intento sospechoso en query_audit_logs.
- La respuesta indica que solo se han usado documentos del presupuesto 484857.

16. Fases de implementacion

Fase	Contenido
1. Infraestructura	PostgreSQL, Qdrant, Redis, redes Docker, volumen /srv/docuintel.

Fase	Contenido
2. Schema y permisos	Migrations, api_clients, permisos por budget_scope, auditoria.
3. Sync seguro	rsync sin borrado, snapshot remoto/local, test no-delete.
4. Ingesta basica	Scanner recursivo, hash, storage por SHA256, PDF digital, Excel, Word, EML.
5. Embeddings y Qdrant	BGE-M3/Nomic, collection, indices payload, query filtrada.
6. API RAG	/api/v1/query, fuentes, redaccion, logs.
7. OCR pesado	PaddleOCR para PDFs escaneados e imagenes.
8. Planos gigantes	Tiles, OCR por zona, coordenadas, VLM en tiles relevantes.
9. Operacion	Panel de jobs, errores, reprocesado, metricas, backups.
10. Integracion	Contrato con la otra app, pruebas E2E, hardening.

17. Decisiones finales

Tema	Decision
BD vectorial	Qdrant self-hosted.
BD de control	PostgreSQL.
Aislamiento	budget_scope_id + API key autorizada + filtro Qdrant obligatorio.
Origen remoto	Solo lectura, nunca borrar ni modificar.
Storage definitivo	files/sha256 por hash.
Planos gigantes	Tiles con solape, no imagen entera.
Embedding potente	BAAI/bge-m3.
IA visual	Qwen2.5-VL 7B.
OCR	PaddleOCR.
Deploy	Coolify + Docker con red privada.

18. Docker Compose base

```

services:
  backend:
    build: ./backend
    environment:
      DATABASE_URL: postgresql://ocr:${POSTGRES_PASSWORD}@postgres:5432/ocr_db
      QDRANT_URL: http://qdrant:6333
      REDIS_URL: redis://redis:6379/0
      EMBEDDING_SERVER_URL: http://embedding-server:8080
    volumes:
      - /srv/docuintel/files:/srv/docuintel/files:ro
    networks: [docuintel-private, docuintel-public]

  worker-fast:
    build: ./workers
    command: celery -A app.celery_app worker -Q scan,extract_fast,embedding,qdrant_upsert -c 2
    volumes:
      - /srv/docuintel:/srv/docuintel:rw
    networks: [docuintel-private]
```

```

worker-plan:
  build: ./workers
  command: celery -A app.celery_app worker -Q plan_heavy,vision_heavy -c 1
  volumes:
    - /srv/docuintel:/srv/docuintel:rw
  networks: [docuintel-private]

```

```

postgres:
  image: postgres:16-alpine
  networks: [docuintel-private]

```

```

qdrant:
  image: qdrant/qdrant:latest
  volumes:
    - qdrant_data:/qdrant/storage
  networks: [docuintel-private]

```

```

redis:
  image: redis:7-alpine
  networks: [docuintel-private]

```

```

networks:
  docuintel-private:
    internal: true
  docuintel-public:

```

19. Mejoras incorporadas para produccion

Esta version refuerza la arquitectura con controles adicionales para evitar fuga de contexto, mejorar la calidad de busqueda y hacer el sistema operable en produccion.

Mejora	Motivo	Aplicacion
Sesiones firmadas por presupuesto	Evita que la otra app cambie manualmente el presupuesto en cada request.	POST /api/v1/sessions devuelve un token ligado a budget_scope_id y con caducidad.
Wrapper unico de Qdrant	Evita consultas vectoriales sin filtro de seguridad.	Todo acceso a Qdrant pasa por search_chunks_for_budget().
Busqueda hibrida	Los presupuestos tienen referencias, habitaciones, medidas y codigos exactos.	Combinar Qdrant semantico + busqueda textual exacta + fusion de resultados.
Reranker	Reduce chunks irrelevantes cuando hay muchos documentos parecidos.	Qdrant top 30 -> reranker -> top 8 final.
Modo no encontrado	Evita respuestas inventadas si no hay evidencia suficiente.	Si score/confianza baja, responder que no se encontro informacion.
Revision humana	OCR y planos pueden fallar.	Panel de needs_review, fallidos, baja confianza y reprocesado.
Versionado documental	Las carpetas pueden cambiar con planos V1/V2 o presupuestos revisados.	document_versions y budget_versions.
Indice espacial para planos	Las medidas y estancias necesitan coordenadas.	plan_pages, plan_tiles, plan_ocr_blocks, dimensiones y habitaciones detectadas.
Backups con prueba de restore	Backup no probado no es backup real.	PostgreSQL, Qdrant snapshots, files/sha256 y prueba mensual.
Observabilidad	Permite detectar colas atascadas, OCR caido o disco lleno.	Metricas, logs, alertas y panel operativo.

19.1 Sesiones firmadas por presupuesto

Mejora recomendada: la otra app no debería enviar libremente X-Presupuesto-Id en cada consulta. Primero solicita una sesión para un presupuesto; el backend valida permisos y devuelve un token firmado.

```
POST /api/v1/sessions
X-API-Key: xxxxx
```

```
Request:
{
  "presupuesto_id": "245745"
}
```

```
Response:
{
  "session_token": "token-firmado",
  "presupuesto_id": "245745",
  "expires_in": 3600
}
```

```
Uso posterior:
POST /api/v1/query
Authorization: Bearer token-firmado
```

Campo del token	Uso
api_client_id	Cliente/API key que creo la sesión.
budget_scope_id	Scope fijo del presupuesto autorizado.
expires_at	Caducidad corta, por ejemplo 1 hora.
permissions	can_query, can_see_amounts, modo debug si aplica.

19.2 Wrapper unico para Qdrant

Ningún módulo debe llamar directamente al cliente Qdrant. El acceso debe pasar por una función interna que exige `budget_scope_id`.

```
def search_chunks_for_budget(budget_scope_id, query_vector, top_k=8):
    if not budget_scope_id:
        raise SecurityError("budget_scope_id obligatorio")

    return qdrant.search(
        collection_name="docu_intel_chunks",
        query_vector=query_vector,
        query_filter={
            "must": [
                {"key": "budget_scope_id", "match": {"value": str(budget_scope_id)}}
            ]
        },
        limit=top_k
    )
```

Regla de test

Toda búsqueda vectorial sin `budget_scope_id` debe fallar automáticamente en tests.

19.3 Búsqueda híbrida + reranker

La búsqueda semántica sola no basta para documentos técnicos. Hay que combinar similitud semántica con búsqueda exacta por referencias, estancias, importes y medidas.

```
Pregunta usuario
-> embedding de consulta
-> Qdrant top 30 dentro del presupuesto
```

- > busqueda exacta en PostgreSQL/FTS: codigos, medidas, habitaciones, referencias
- > fusion de candidatos
- > reranker multilingual
- > top 8 final
- > IA responde con fuentes

Tipo de consulta	Mejor tecnica
"habitacion 203"	Busqueda textual exacta + entidades.
"motor Somfy"	Texto exacto + semantica.
"que partidas son de cortinas"	Semantica + reranker.
"ancho 3200"	Entidad dimension + OCR blocks.
"presupuesto revisado"	Versionado + metadatos.

19.4 Modo no encontrado y control de alucinaciones

La IA debe poder decir que no hay informacion suficiente. No debe rellenar huecos con inferencias no soportadas.

Si no hay chunks relevantes:

```
{
  "respuesta": "No he encontrado informacion suficiente dentro del presupuesto 245745.",
  "confidence": "low",
  "fuentes": [],
  "advertencias": [
    "La busqueda se ha limitado al presupuesto 245745."
  ]
}
```

- Definir score minimo de recuperacion.
- Definir minimo de fuentes validas para responder.
- Separar respuestas confirmadas de inferencias.
- Obligar a citar documento, pagina, tile o chunk.

19.5 Revision humana y versionado

Modulo	Contenido
Revision humana	Documentos con baja confianza, OCR fallido, planos dudosos, chunks vacios, duplicados y errores.
Reprocesado	Reprocesar OCR, embeddings, entidades o Qdrant sin repetir todo el presupuesto.
Versionado	Distinguir plano V1/V2, presupuesto revisado, factura rectificativa o archivo reemplazado.
Trazabilidad	Indicar en cada respuesta que version documental se ha usado.

```
CREATE TABLE document_versions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  document_id UUID NOT NULL REFERENCES documents(id),
  version_number INTEGER NOT NULL,
  sha256 VARCHAR(64) NOT NULL,
  storage_path TEXT NOT NULL,
  is_current BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW()
);
```

```
CREATE TABLE budget_versions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  budget_scope_id UUID NOT NULL REFERENCES budget_scopes(id),
  version_number INTEGER NOT NULL,
  description TEXT,
```

```

created_at TIMESTAMP DEFAULT NOW()
);

```

19.6 Indice espacial para planos

Para planos gigantes no basta guardar texto. Hay que guardar donde aparece cada texto, medida o estancia.

Tabla	Datos
plan_pages	Pagina, ancho, alto, DPI, preview.
plan_tiles	Tile, x, y, width, height, overlap, path.
plan_ocr_blocks	Texto, bbox, confianza, tipo de bloque.
plan_detected_dimensions	Medida visible, unidad, bbox, confianza.
plan_detected_rooms	Estancia/habitacion detectada, bbox, confianza.

Regla de medidas

Solo se afirman medidas escritas en el plano o extraidas tecnicamente. No calcular distancias geometricas con IA si no hay escala fiable.

19.7 Backups, restore y observabilidad

Area	Requisito minimo
PostgreSQL	Backup diario y prueba mensual de restauracion.
Qdrant	Snapshots programados y restauracion en entorno de prueba.
files/sha256	Backup incremental con restic/rsync a storage externo.
Secretos	.env y claves fuera del repositorio.
Metricas	Colas, OCR, chunks, fallos, tiempos, disco, consultas.
Alertas	Disco >80%, Qdrant caido, Postgres caido, worker parado, cola alta.

Panel operativo minimo:

- presupuestos totales / procesados / fallidos
- archivos pendientes / procesados / con error
- chunks generados
- errores OCR por tipo
- cola Celery por worker
- uso de disco
- tamano PostgreSQL y Qdrant
- consultas por API key
- consultas bloqueadas o sospechosas

19.8 Prioridad de implementacion de las mejoras

- Sesiones firmadas por presupuesto.
- Wrapper unico para Qdrant.
- Tests de aislamiento.
- Modo no encontrado.
- Reranker.
- Busqueda hibrida.
- Revision humana.
- Versionado documental.
- Indice espacial para planos.
- Backups, restore y observabilidad.

20. Referencias técnicas

- Qdrant repository - Apache License 2.0: <https://github.com/qdrant/qdrant>
- Qdrant Cloud free tier: <https://qdrant.tech/documentation/cloud/create-cluster/>
- Qdrant payload filtering: <https://qdrant.tech/documentation/search/filtering/>
- Qdrant multitenancy via payload: <https://qdrant.tech/documentation/manage-data/multitenancy/>
- BAAI/bge-m3 model card: <https://huggingface.co/BAAI/bge-m3>
- Qwen2.5-VL on Ollama: <https://ollama.com/library/qwen2.5vl>
- Qwen2.5-VL official blog: <https://qwenlm.github.io/blog/qwen2.5-vl/>
- PaddleOCR repository: <https://github.com/PaddlePaddle/PaddleOCR>
- Jina Embeddings v4 model card: <https://huggingface.co/jinaai/jina-embeddings-v4>